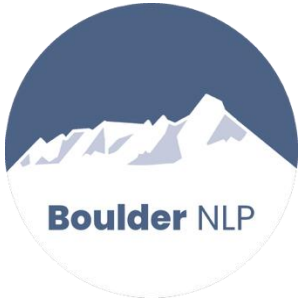
Natural Language Processing

Jim Martin -- CSCI 5832

Lecture 13



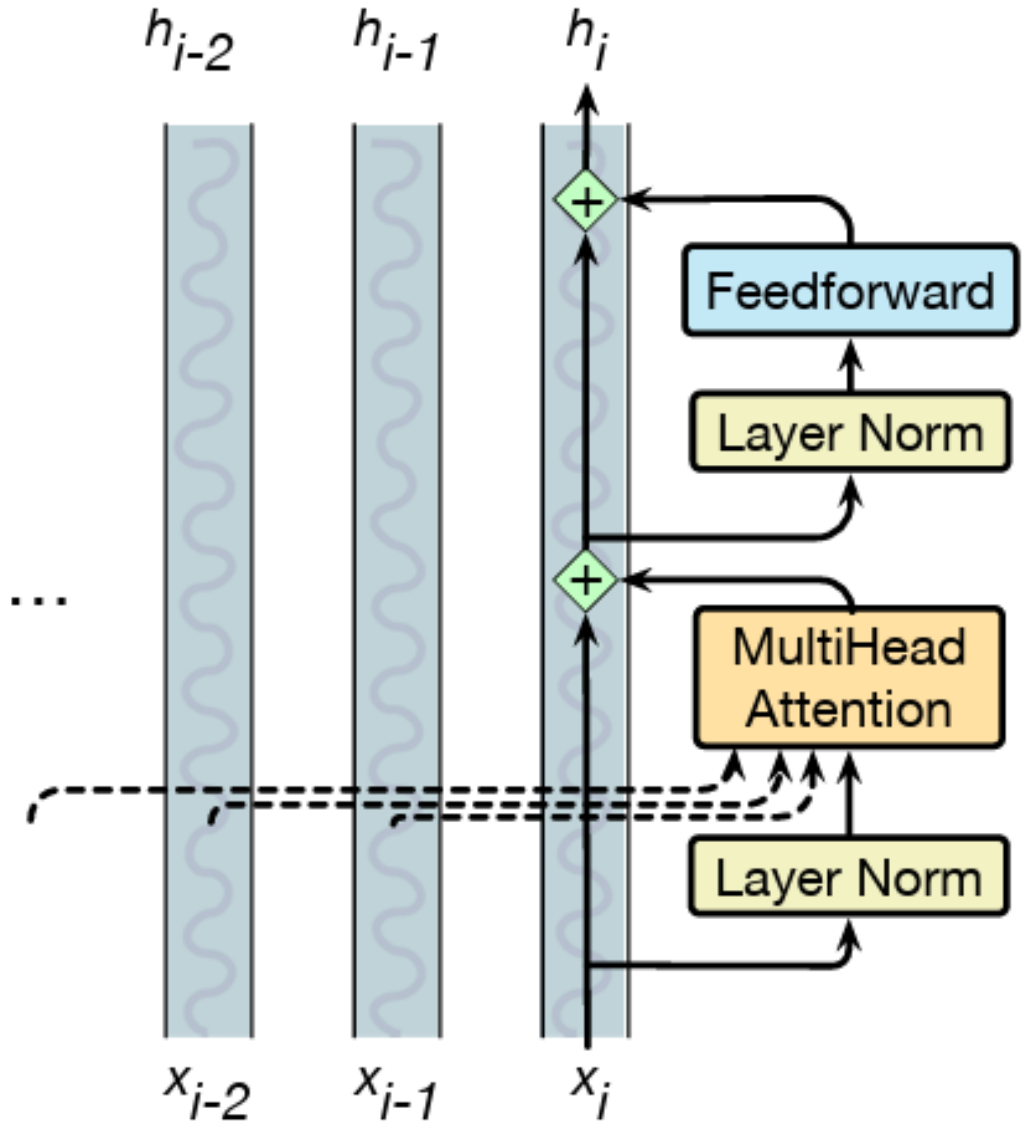
Today

SemEval Tasks and Teams

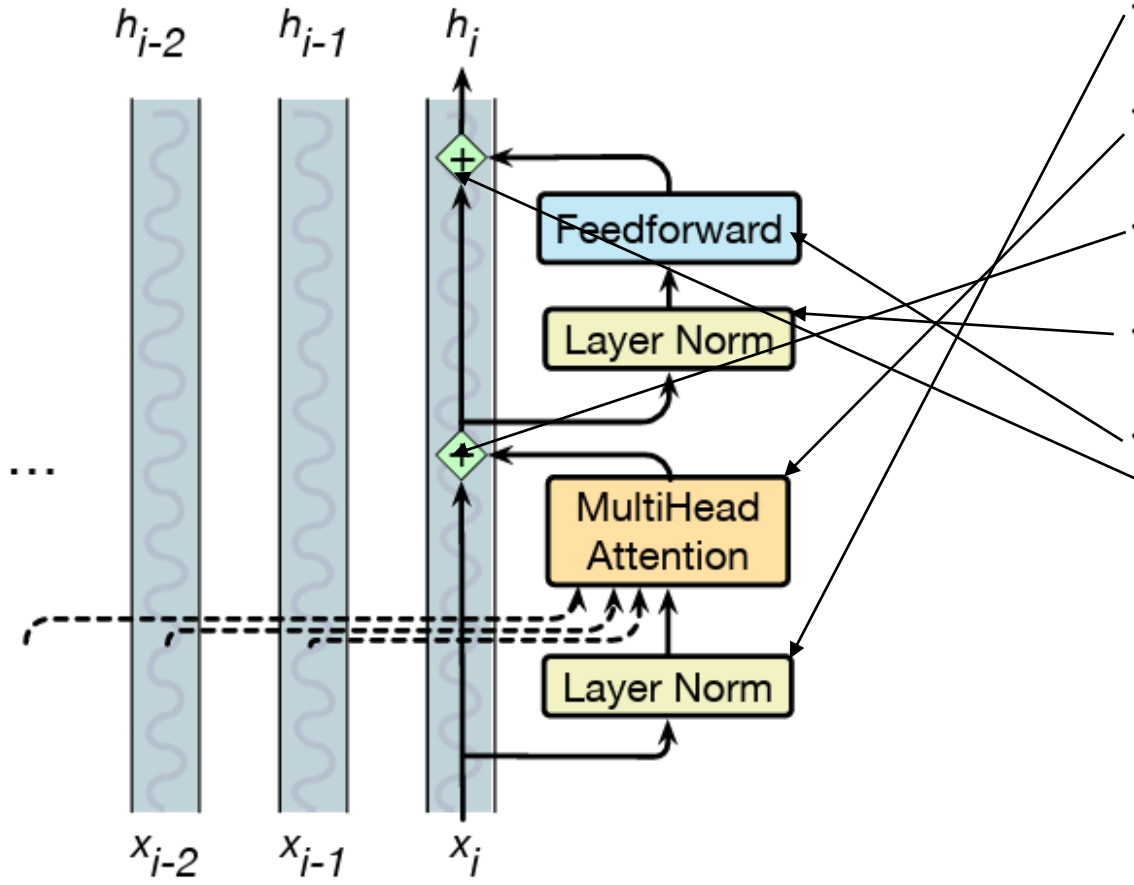
- Form teams and choose tasks
  - 2-3 Person teams
    - Choose wisely
  - Options are Tasks 1-5, 7, 8, and 9.1
- Go to Canvas > People > SemEval Project Teams and assign yourself to your team. Pick whatever team # you like.
- Final Submission is due on 12/7 5PM
  - No presentations

Then more transforms/training/decoding.

Transformer Blocks



Transformer Block



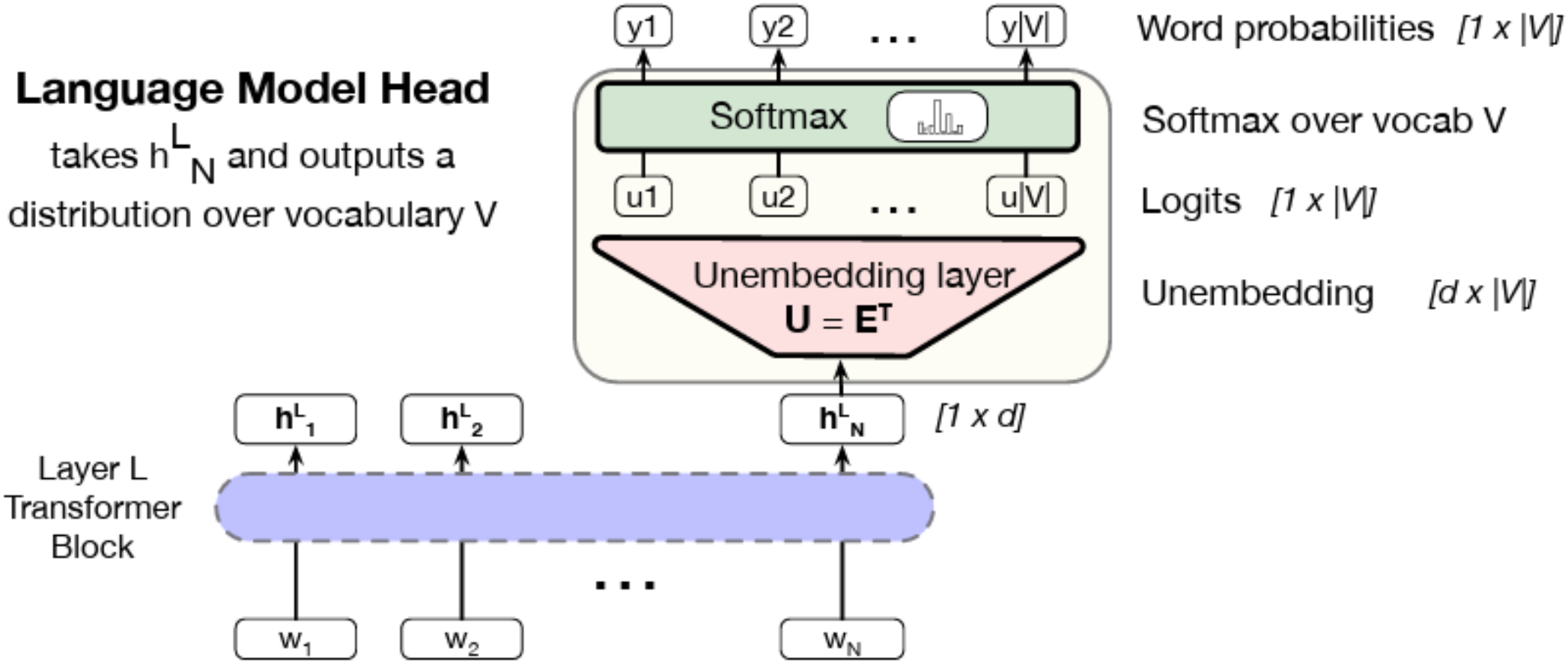
$$
\begin{aligned}
\mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{t}_1^1, \cdots, \mathbf{t}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

Model Size Example

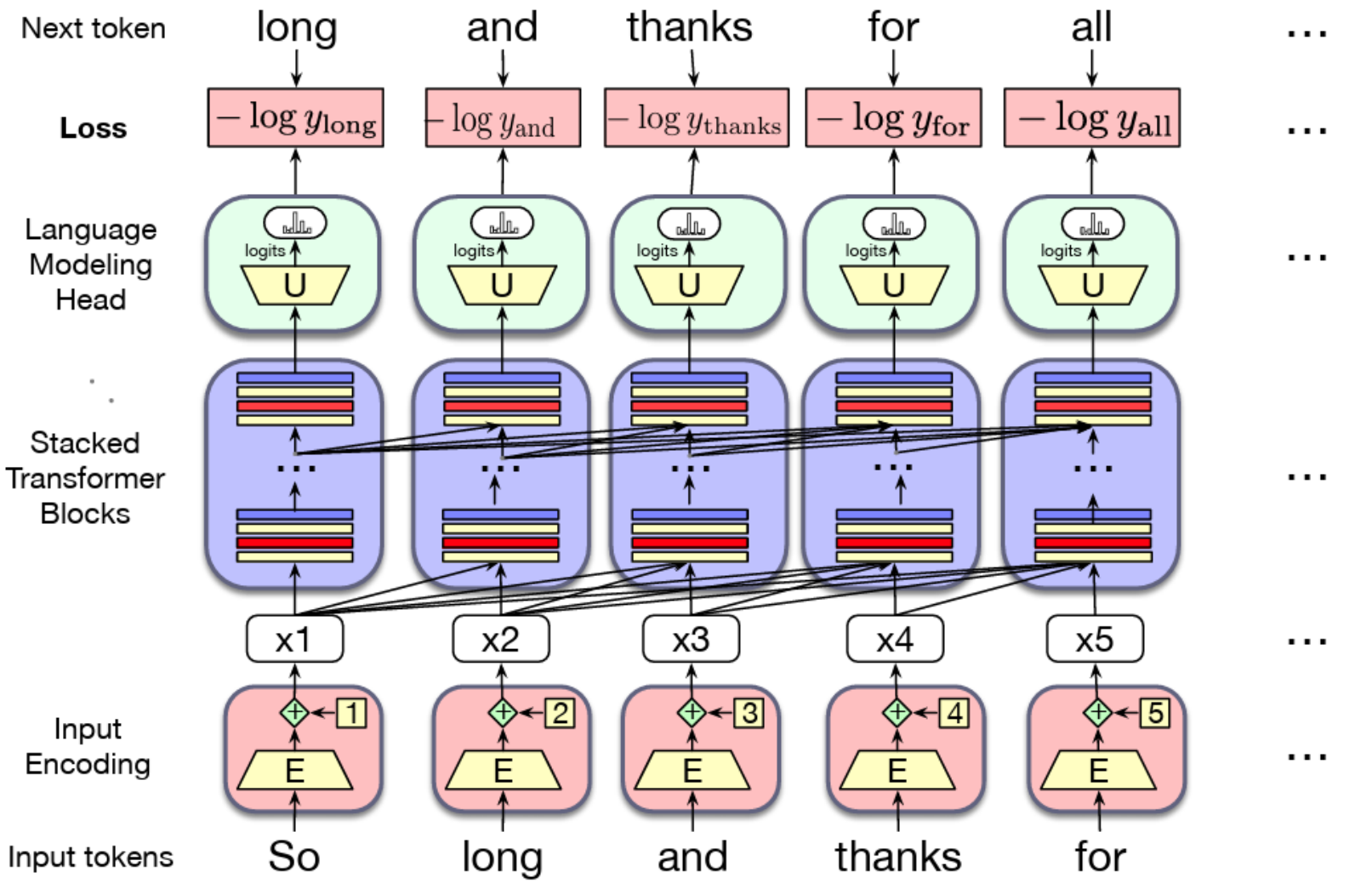
GPT 3 (175B)

- 50k BPE vocabulary
- Embedding layer/ model dimensionality: 12,288
  - Meaning embedding matrix E is… (50k x 12k) = ~600M parameters
  - As is the language modeling head U
- 96 Transformer blocks (layers)
  - 96 attention heads per block (multihead attention)
  - FF hidden layers 49k (4x the model dim)

Transformer as Language Model: The Language Model Head

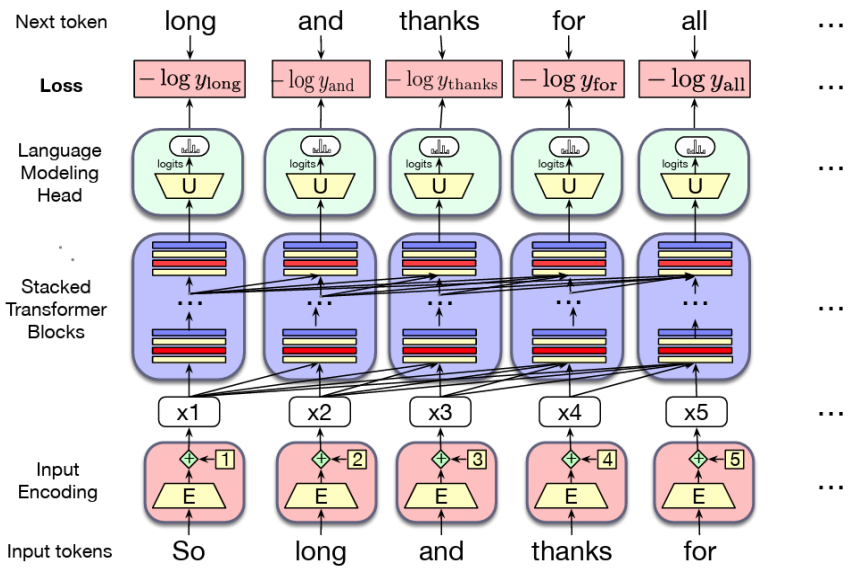


Transformer Language Model Training



$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

Transformer Language Model Training



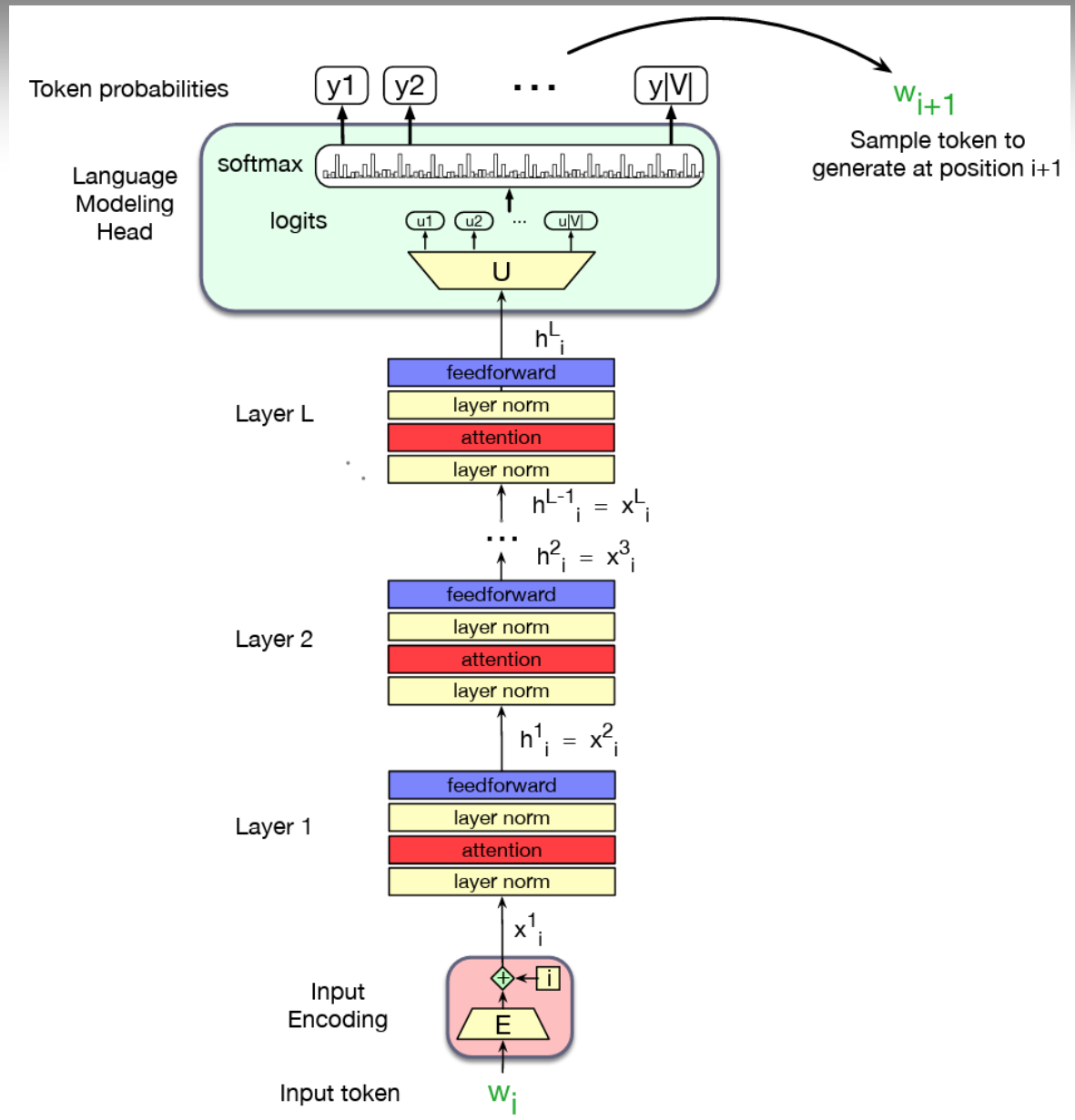
$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

No temporal dependence here. Given a training text, we know the right answer for each point in time. And each of those answers is independent of all the other answers (the forward computation for each time step is independent of the other time steps).

- As with other gradient-descent methods, we don't want to do a weight update on each item. We can use a minibatch-based approach. Accumulate a loss over a batch, average it, and use that for a single weight update.
  - This is a huge advantage over RNN training which is inherently serial due to the recurrence.

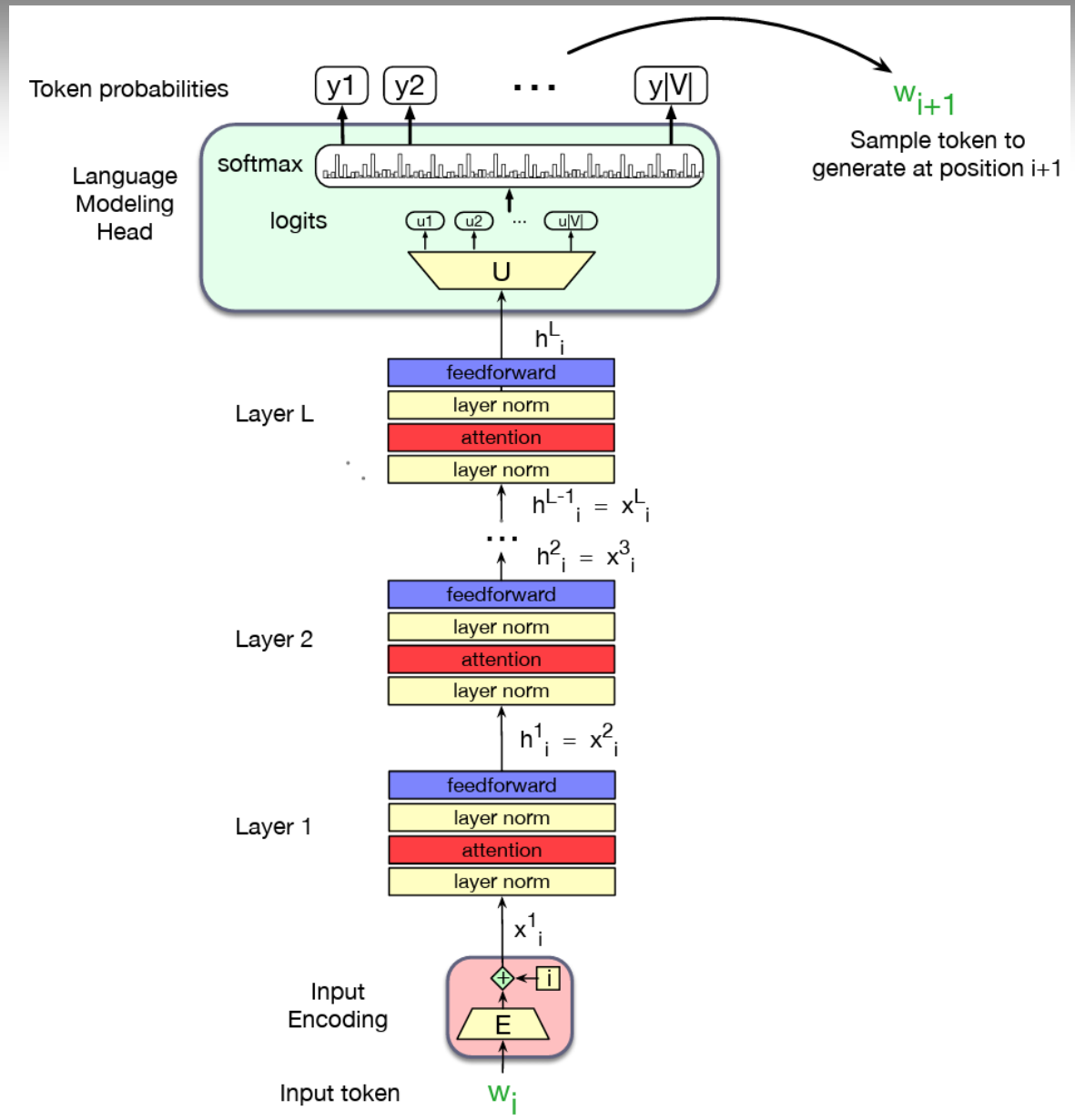
Inference (Test-time) Compute

But what about forward inference?



Inference (Test-time) Compute

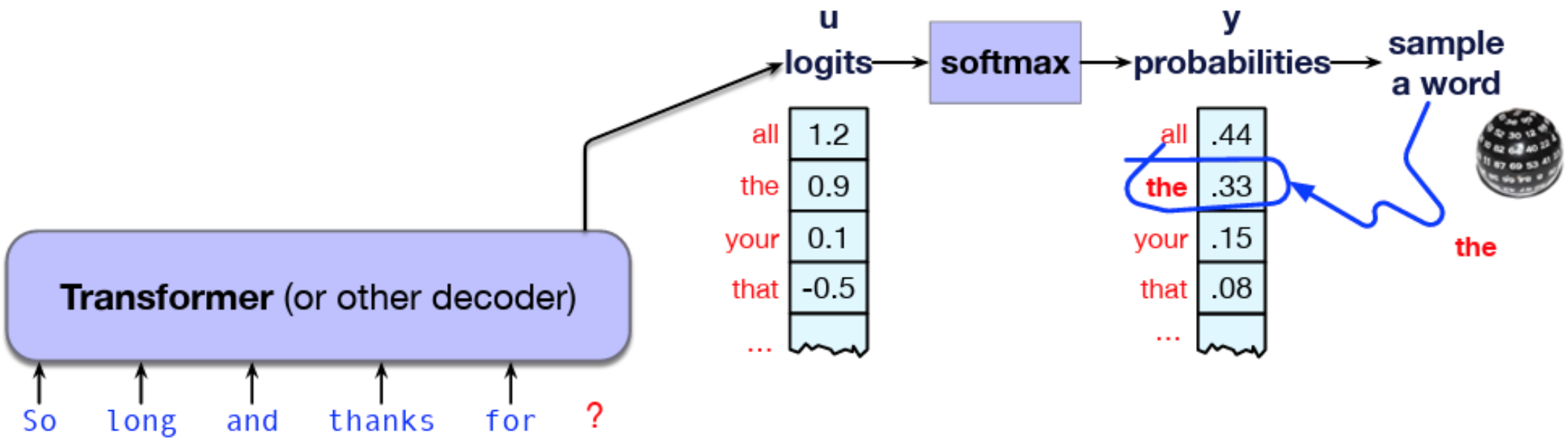
Inference based on autoregressive decoding is inherently serial. You can't decode at time i+1 without first having sampled to get the output at time i.



Sampling

- Greedy
  - Take the top choice based on the distribution (argmax decoding).
- Random sampling from the distribution
- Temperature sampling
- Top-K sampling
- Nucleus (Top-p) sampling

Random Sampling



$$y = \text{softmax}(u)$$

Random Sampling

“One fish two fish red fish blue fish black fish blue fish”

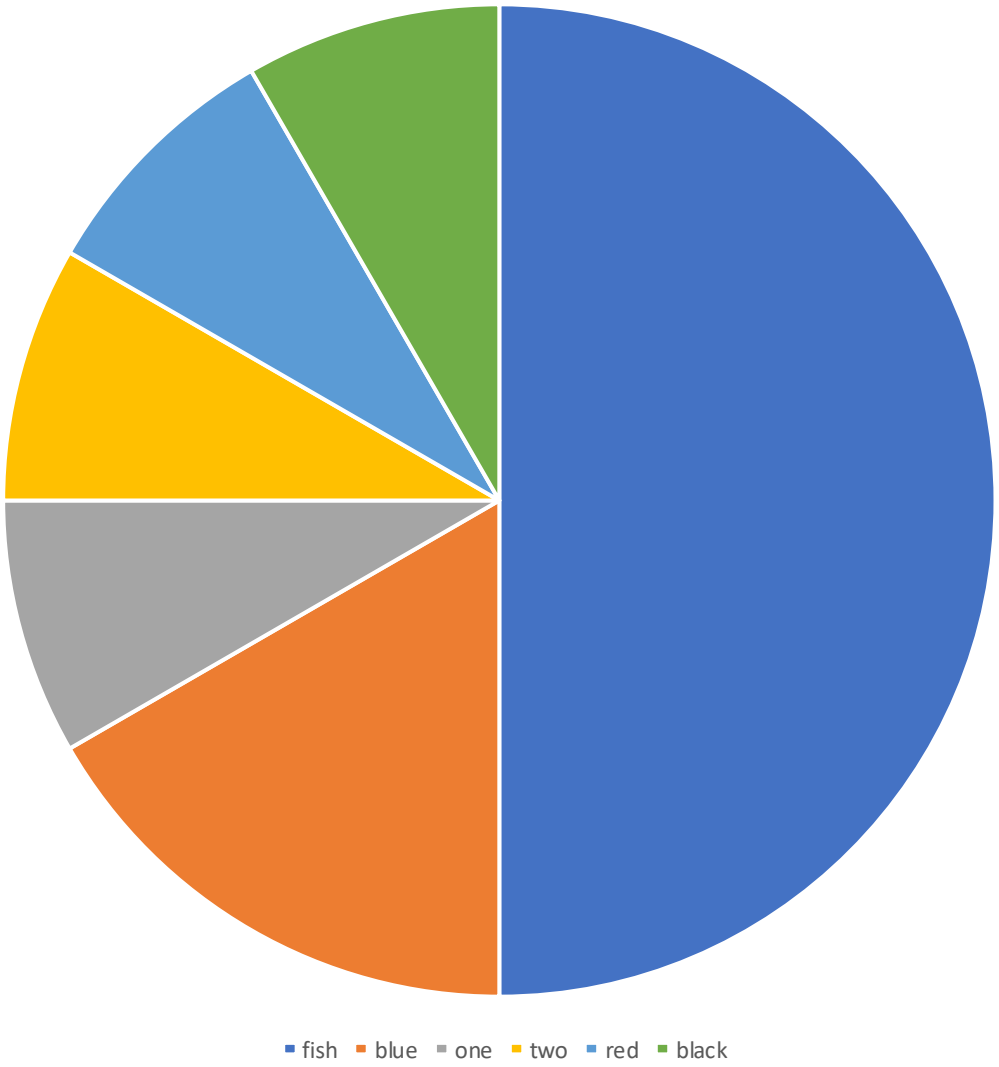
$N = 12$

$|V| = 6$

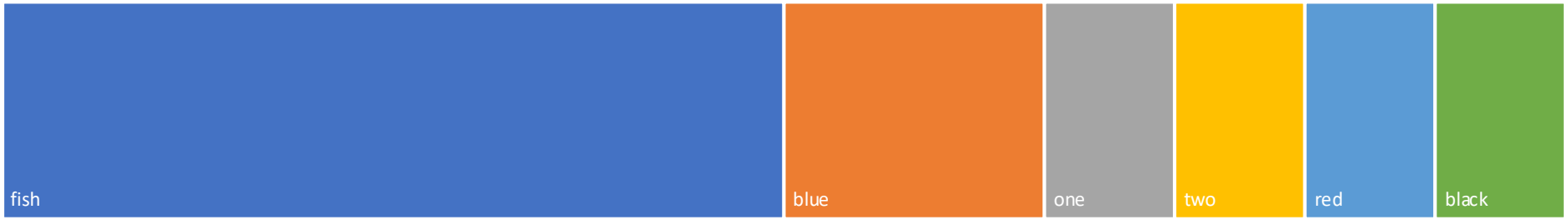
Random Sampling

- Counts
  - ”fish” = 6
  - “blue” = 2
  - ”one” = 1
  - “two” = 1
  - ”red” = 1
  - “black” = 1
- Probs
  - ”fish” = 6/12
  - “blue” = 2/12
  - ”one” = 1/12
  - “two” = 1/12
  - ”red” = 1/12
  - “black” = 1/12

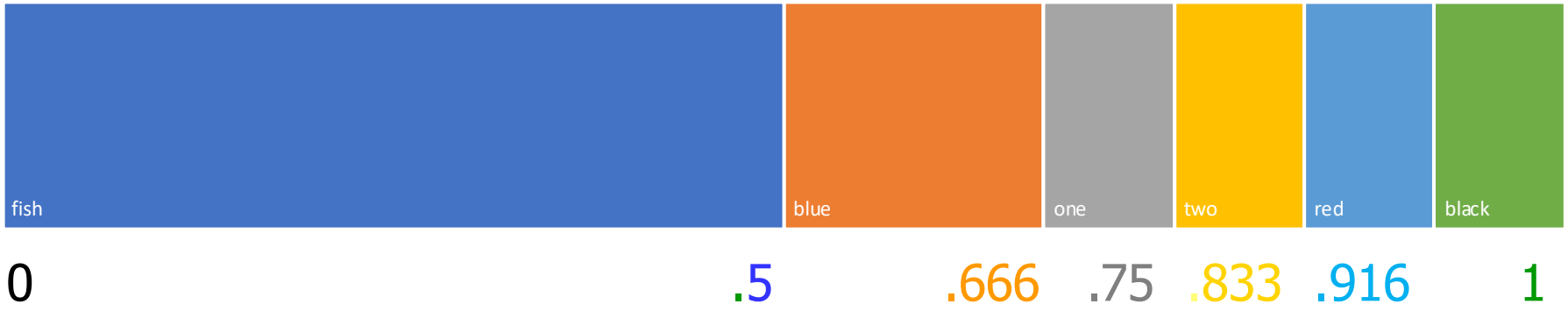
Random Sampling



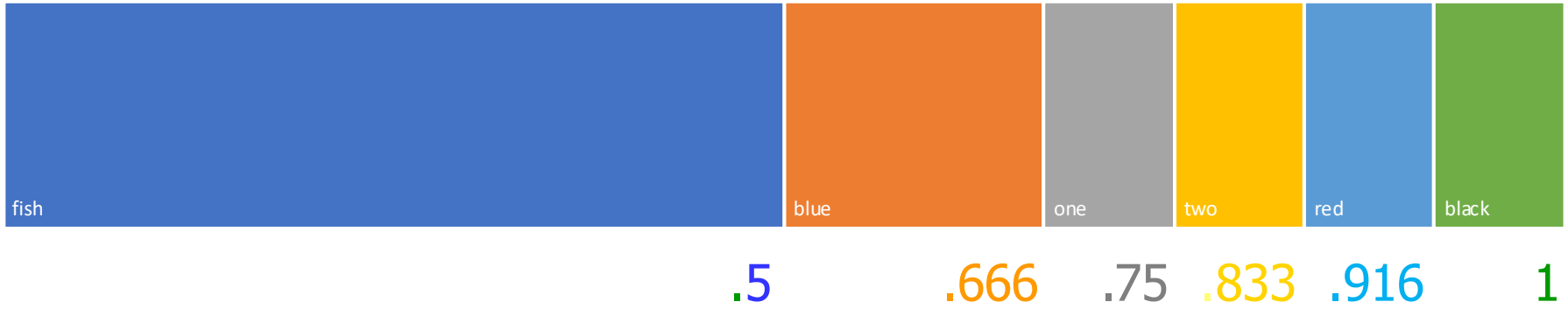
Random Sampling



Random Sampling



Random Sampling



Generate a random number between 0 and 1
Find the bin that it falls in, return the word associated with that bin
Repeat

Numpy Version

In [30]:
```
1 sample
```

Out[30]:
```
array(['fish', 'fish', 'one', 'blue', 'fish', 'fish', 'fish', 'fish',
       'one', 'one', 'blue', 'two', 'fish', 'blue', 'fish', 'fish', 'two',
       'blue', 'black', 'blue', 'fish', 'fish', 'blue', 'blue', 'fish',
       'fish', 'fish', 'fish', 'blue', 'two', 'fish', 'fish', 'one',
       'blue', 'fish', 'red', 'blue', 'fish', 'fish', 'fish', 'blue',
       'fish', 'fish', 'one', 'blue', 'red', 'red', 'blue', 'blue',
       'black', 'fish', 'fish', 'red', 'one', 'fish', 'black', 'black',
       'red', 'blue', 'fish', 'black', 'one', 'blue', 'fish', 'blue',
       'fish', 'one', 'red', 'red', 'blue', 'fish', 'blue', 'fish',
       'black', 'two', 'fish', 'fish', 'blue', 'fish', 'fish', 'black',
       'fish', 'fish', 'one', 'fish', 'fish', 'two', 'two', 'fish',
       'fish', 'fish', 'two', 'black', 'fish', 'blue', 'red', 'blue',
       'black', 'two', 'two'], dtype='<U5')
```

In [31]:
```
1 np.unique(weighted_sample, return_counts=True)
2
```

Out[31]:
```
(array(['black', 'blue', 'fish', 'one', 'red', 'two'], dtype='<U5'),
 array([ 5, 18, 53,  9,  8,  7]))
```

Problem

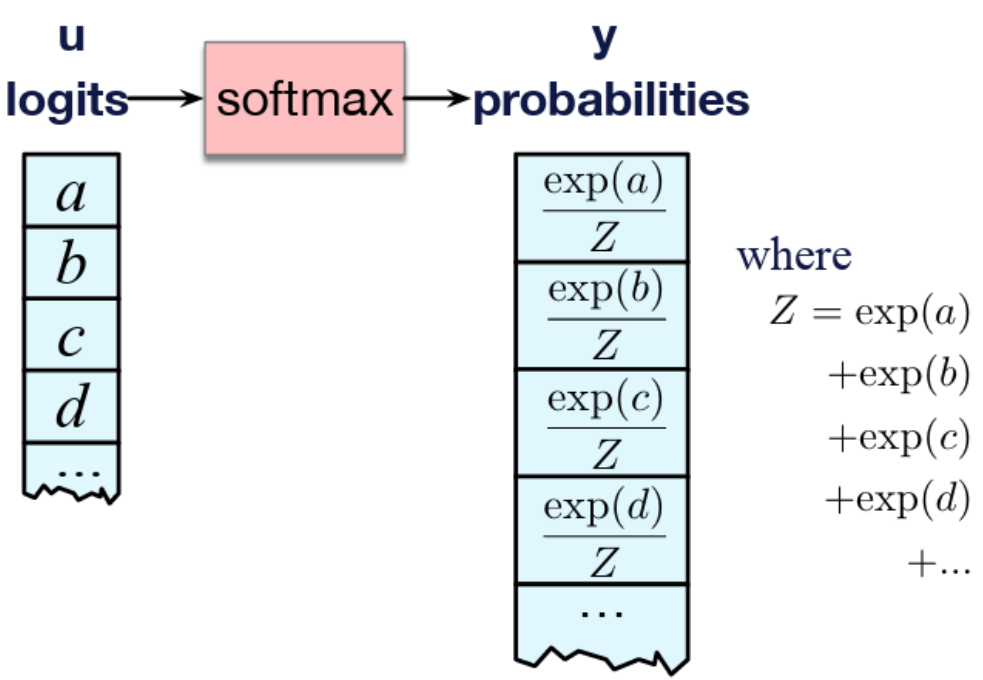
That approach produces some odd outputs since the distribution is so skewed.

Better approaches try to fix this by manipulating the distribution or restricting the sample space.

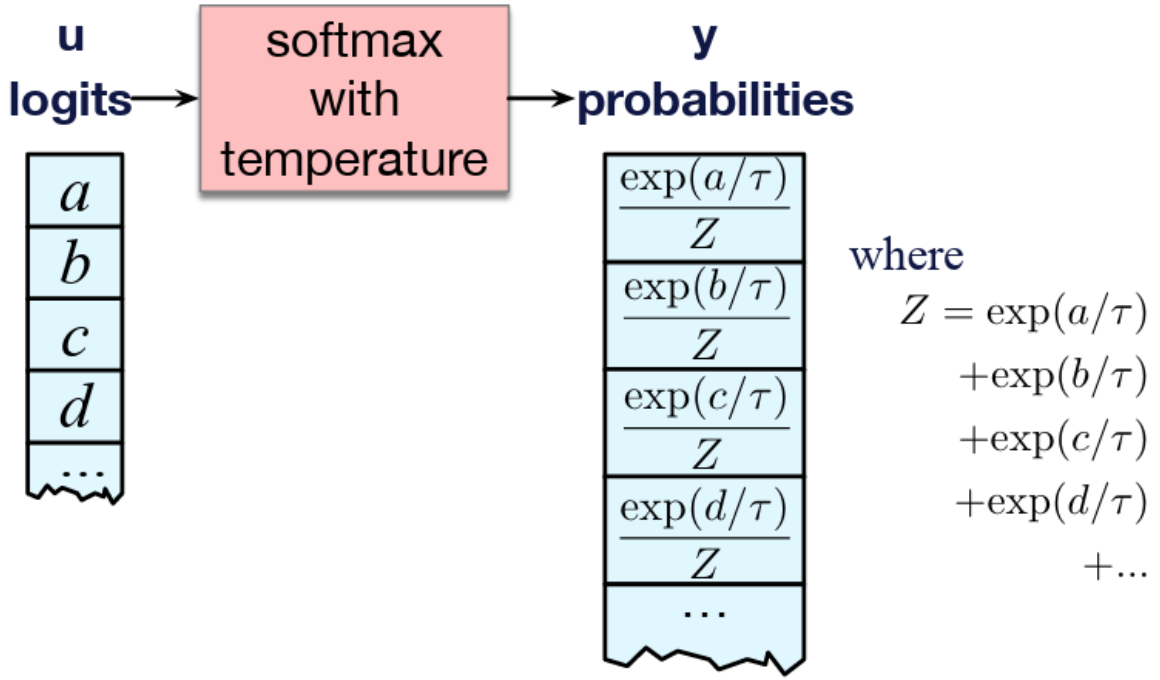
- Temperature sampling
- Top-k sampling
- Top-p sampling

Temperature

- Divide the logit by a “temperature” > 0, typically between 0 and 1.

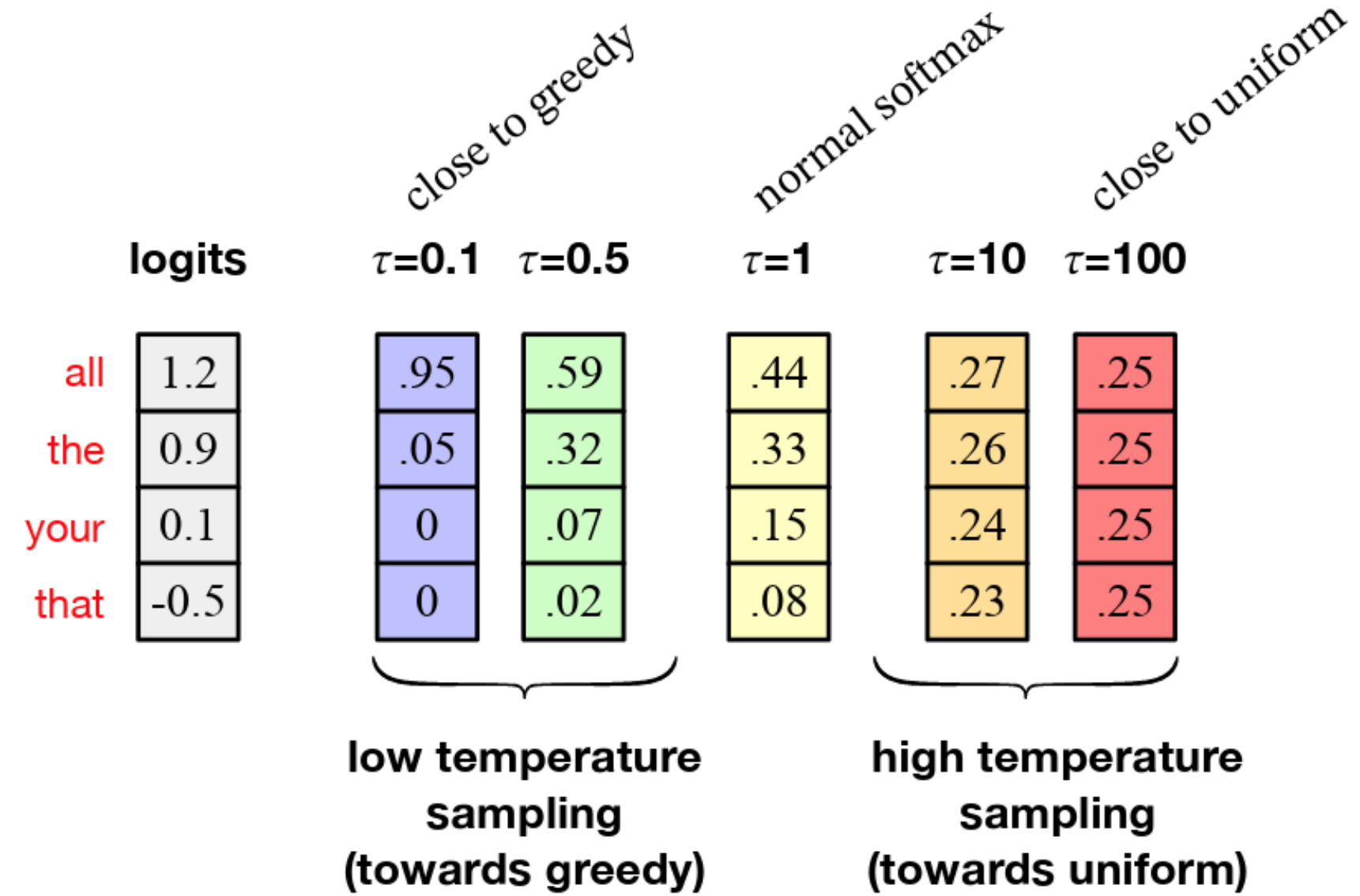


(a) $\mathbf{y} = \mathrm{softmax}(\mathbf{u})$



(b) $\mathbf{y} = \mathrm{softmax}(\mathbf{u}/\tau)$

Temperature



Top-k

1. Truncate the original V to just the top-k logits
2. Generate softmax from that
3. Sample from that distribution

Never generates a word outside the top-k

But the top-k differs with each time step

Top-p

- Only keep the top p percent of the probability mass and sample from that. So instead of the sampling from the top-k terms, sample from for example, the top 50% of the probability mass.

$$\sum_{w \in V^{(p)}} P(w|\mathbf{w}_{<t}) \geq p$$